

```
"""
```

```
uma/venturi/injector.py - CrossDomainInjector
```

Couples GENERIC + MSR through the 360-degree evolving Venturi.

Full coupled differential:

```
GENERIC:   $\partial_t \psi = -\delta H / \delta \psi^* + \sqrt{(2kT \cdot \lambda)} \cdot \xi + G \cdot V(r, r_0) \cdot \phi_{\text{inject}}$   
MSR:       $\hat{\psi} = -dH / d\psi^*$  (exact response field from GENERIC)  
Throat:    $dr_0 / dt = -\lambda_v \cdot (E - E^*) / E^* \cdot r_0$ 
```

Both domains flow through the same $V(r, r_0(t))$.

Semantic injection:

```
text → binary weight x  
x → z_c = x · ((1-√2) + i√e)    [Silver seed]  
z_c →  $\phi_s(x, y) = |z_c| \cdot \exp(i \cdot (\arg(z_c) + \theta(x, y)))$  [360° azimuthal]
```

MSR injection:

```
 $\hat{\psi} = -dH / d\psi^*$  evaluated at current  $\psi \rightarrow$  injected via Venturi
```

```
"""
```

```
from __future__ import annotations
```

```
import math
```

```
import numpy as np
```

```
from typing import Optional, Tuple, List
```

```
from uma.venturi.operator import VenturiOperator, VenturiConfig, VenturiState
```

```
from uma.semantic.constants import (
```

```
    engine3_complex_state, INV_DELTA_S_SQ,
```

```
    silver_E_target, silver_field_scale,
```

```
)
```

```
class CrossDomainInjector:
```

```
    """
```

Couples GENERIC + MSR through the 360-degree evolving Venturi.

Usage::

```
inj = CrossDomainInjector(N=32, uma_cfg=cfg)
```

```
# Semantic injection (text → field)
```

```
psi_out, state = inj.inject_semantic("input text", psi, z_dc, dt)
```

```

# MSR injection (response field → GENERIC)
psi_out, state = inj.inject_msr(psi, psi_hat, z_dc, dt)

# Full differential (both domains simultaneously)
psi_out, state = inj.inject(psi, psi_hat, z_dc, dt, text="input")

# Run n steps against a live UMAClient
result = inj.run(client, n_steps=100, dt=0.04, text="input")
"""

def __init__(
    self,
    N: int,
    uma_cfg=None,
    venturi_cfg: Optional[VenturiConfig] = None,
):
    self.N = N
    self._kT = uma_cfg.generic.kT if uma_cfg else 0.005

    vcfg = venturi_cfg or (
        VenturiConfig.from_uma_config(uma_cfg) if uma_cfg else VenturiConfig()
    )
    self.venturi = VenturiOperator(N, vcfg)

# -----
# Semantic injection
# -----

def inject_semantic(
    self,
    text: str,
    psi: np.ndarray,
    z_dc: np.ndarray,
    dt: float,
    anchor: str = "dIse",
) -> Tuple[np.ndarray, VenturiState]:
    """
    text → binary weight x → Silver complex z_c → 360° field → Venturi
    """
    binary = ''.join(format(ord(c), '08b') for c in text)
    anc_b = ''.join(format(ord(c), '08b') for c in anchor)
    x = len(binary) / max(len(anc_b), 1)
    z_c = engine3_complex_state(x)
    phi_s = self._azimuthal_field(z_c, psi)
    return self.venturi.step(psi, phi_s, z_dc, dt)

```

```

def _azimuthal_field(self, z_c: complex, psi: np.ndarray) -> np.ndarray:
    """
    360° injection field:  $\phi(x,y) = |z_c| \cdot \exp(i \cdot (\arg(z_c) + \theta(x,y)))$ 
     $\theta(x,y)$  = azimuthal angle from field center of mass.
    U(1) gauge invariant – isotropic in all directions.
    """
    intensity = np.abs(psi)**2
    total      = intensity.sum() + 1e-15
    coords     = np.linspace(0, 1, self.N, endpoint=False)
    X, Y       = np.meshgrid(coords, coords)
    cx         = float((intensity * X).sum() / total)
    cy         = float((intensity * Y).sum() / total)
    dx = X - cx; dx -= np.round(dx)
    dy = Y - cy; dy -= np.round(dy)
    theta = np.arctan2(dy, dx)
    return abs(z_c) * np.exp(1j * (np.angle(z_c) + theta))

# -----
# MSR injection
# -----

def inject_msr(
    self,
    psi: np.ndarray,
    psi_hat: np.ndarray,
    z_dc: np.ndarray,
    dt: float,
) -> Tuple[np.ndarray, VenturiState]:
    """
     $\hat{\psi}$  (MSR response field)  $\rightarrow$  Venturi  $\rightarrow$  injected into  $\psi$ .

    In full theory:  $\hat{\psi} = -dH/d\psi^*$  (from client.dynamics.dH_dpsi_conj).
    Pass that directly for exact coupling.
    """
    return self.venturi.step(psi, psi_hat, z_dc, dt)

# -----
# Full differential (both domains)
# -----

def inject(
    self,
    psi: np.ndarray,
    psi_hat: np.ndarray,
    z_dc: np.ndarray,
    dt: float,
    text: Optional[str] = None,

```

```

        anchor: str = "dIse",
    ) -> Tuple[np.ndarray, VenturiState]:
        """
        Full cross-domain coupling differential.
         $\phi_{\text{total}} = \hat{\psi}_{\text{MSR}} + \phi_{\text{semantic}}$  (if text provided)
        Both flow through the same Venturi  $V(r, r_0(t))$ .
        """
        phi = psi_hat.copy()

        if text is not None:
            binary = ''.join(format(ord(c), '08b') for c in text)
            anc_b   = ''.join(format(ord(c), '08b') for c in anchor)
            x       = len(binary) / max(len(anc_b), 1)
            phi     = phi + self._azimuthal_field(engine3_complex_state(x), psi)

        return self.venturi.step(psi, phi, z_dc, dt)

# -----
# Run against a live UMAClient
# -----

def run(
    self,
    client,
    n_steps: int,
    dt: float,
    text: Optional[str] = None,
    return_history: bool = False,
) -> dict:
    """
    Run the full coupled system for n_steps against a live UMAClient.

    Each step:
    1. GENERIC dynamics step
    2.  $\hat{\psi} = -dH/d\psi^*$  (exact MSR response field)
    3. Venturi injects  $\hat{\psi} + \text{semantic field}$  into  $\psi$ 
    4. Update posterior
    """
    from uma.core.state import FieldPosterior

    self.venturi.reset()
    posterior = client.posterior_state
    history   = [] if return_history else None
    state     = None

    for step in range(n_steps):
        # 1. GENERIC

```

```

psi    = client.projection.lift(posterior.mean)
psi    = client.dynamics.step(psi, dt, rng=client.rng)
z_dc   = client.projection.project(psi)

# 2. Exact MSR response field:  $\hat{\psi} = -dH/d\psi^*$ 
psi_hat = -client.dynamics.dH_dpsi_conj(psi)

# 3. Venturi injection (both domains)
psi_out, state = self.inject(psi, psi_hat, z_dc, dt, text=text)

# 4. Update
z_new      = client.projection.project(psi_out)
posterior = FieldPosterior.full(z_new, posterior.Sigma)
client.posterior_state = posterior
client.t += dt

if return_history:
    history.append({
        "step": step, "t": state.t,
        "r0": state.r0, "G": state.G,
        "E": state.E_field, "dr0_dt": state.dr0_dt,
        "z": z_new.copy(),
    })

return {
    "final_state": state,
    "final_z":     z_dc,
    "r0_final":    state.r0 if state else None,
    "G_final":     state.G  if state else None,
    "history":     history,
}

@property
def state(self) -> Optional[VenturiState]:
    return self.venturi.state

```